EC327 MIDTERM 1 HANDBOOK

midterm1 cheat pdf

Dense, readable, exam-oriented quick reference for syntax repair, functions, headers, arrays, file I/O, pointers, heap memory, and diagram-to-code questions.

Built from your notes, code examples, practice-midterm skeleton files, and selected worksheets.

Design goal: fast lookup under time pressure, not a raw note dump.

Best use: recognize the pattern, copy the right syntax, then debug with the checklists.

Weights used: Pointers 40% · Functions / headers / prototypes 30% · Arrays + file I/O 20% · Basic syntax / control flow 10%

You said recursion is not on this midterm, so it is intentionally omitted from the core reference.

Part 0 - Exam survival

Compile-first mindset

If it does not compile, it gets zero.
Get the file structure, function headers, return types, braces, semicolons, and includes correct before chasing logic bugs.

Single-file compile

```
g++ -Wall -std=c++98 file.cpp
./a.out
```

Multi-file compile

```
g++ -Wall -std=c++98 main.cpp timeCalc.cpp
raceCalc.cpp
./a.out
```

Submission skeleton

```
mkdir yourname_0000_000_midterm
cd yourname_0000_000_midterm
# put every required file here
cd ..
zip -r yourname_0000_000_midterm.zip ./
yourname_0000_000_midterm/*
```

Top 12 compile killers

Mistake	Fast fix
Missing `;`	Check the line above the error too.
Wrong quotes	Strings use `"..."`, chars use `'a'`.
<< vs >>	`cout <<`, `cin >>`.
Prototype mismatch	Parameter types, order, const, and return type must match.
Non-void with no return	Every path must return.
Using array size 10 with loop 1..10	Valid indexes are 0..9.
Forgetting header include	`#include "myHeader.h"`
Using `while(!input.eof())`	Prefer `while (input >> temp)`.
Using = inside condition accidentally	Need == for equality test.
Missing `break` in switch	Add one unless fall-through is intended.
Forgetting `delete[]` for heap arrays	`new[]` pairs with `delete[]`.
Trying to change caller pointer with plain `int *`	Use `int *&`.

lookup path Q1 -> syntax / headers / prototypes. **lookup path** Q2 -> arrays / file I/O / function signatures. **lookup path** Q3 -> pointer diagrams / heap / pointer rebinding.

Part 1 - Syntax repair, control flow, and I/O

Minimal C++ skeleton

```
#include <iostream>
using namespace std;

int main() {
    int x;
    cin >> x;
    cout << x << endl;
    return 0;
}
```

Casting and arithmetic

```
cout << 1 / 2 << endl;                    //
0
cout << static_cast<double>(1) / 2 <<
endl; // 0.5

int i = static_cast<int>(3.7); // 3
```

if / else pattern

```
if (answer == 'Y') {
    date += 1;
} else {
    date -= 1;
}
```

for / while pattern

```
for (int i = 0; i < n; i++) {
    // work
}

int i = 0;
while (i < n) {
    // work
    i++;
}
```

switch pattern

```
switch (code) {
    case 'a':
        // statements
        break;
    case 'b':
        // statements
        break;
    default:
        cout << "Error" << endl;
}
```

Quick syntax checks

Headers	`#include <iostream>` or `#include <fstream>`
String vs char	`"hello"` vs `'h'`
Input	`cin >> variable;`
Output	`cout << value << endl;`
Increment	`i++` or `++i`; both legal, different expression value
Scope	Variables exist only inside their block unless global.

Part 2 - Functions, prototypes, headers, and separate compilation

Three function ideas you must keep separate

Concept	What it is	Example
Prototype / declaration	Tells the compiler name, return type, parameter types.	`double calcSplit(int metric, char units, double time);`
Definition	Actual body of the function.	`double calcSplit(...) { ... }`
Call / invoke	Use the function.	`pace = calcSplit(1, 'm', t);`

Prototype decision table

You want...	Parameter form	Example
Read a plain value	pass by value	`int n`, `double gpa`
Modify caller's variable	reference	`double &ave`, `int &size`
Read array contents	pointer / array parameter	`const int data[]`, `const double *const list`
Modify through a pointer target	pointer	`int *p`
Change the caller's pointer itself	reference to pointer	`int *&p`
Return a new heap array	pointer return type	`double *addNewClass(...)`

Header template

```
#ifndef MYLIB_H
#define MYLIB_H

int max(int, int);
void swap(int &, int &);
double *addNewClass(const double *const, const double, int &);

#endif
```

Source file template

```
#include "mylib.h"
#include <iostream>
using namespace std;

int max(int a, int b) {
    return (a > b) ? a : b;
}
```

Pass-by-value vs reference vs pointer

```
void swap1(int a, int b) { /* caller unchanged */ }
void swap2(int &a, int &b) { /* caller changed */ }
void swap3(int *a, int *b) { /* caller values changed via *a and *b */ }
```

Exam trap: if the function must rebind a pointer in the caller, `int *p` is not enough. Use `int *&p`.

Part 3 - Arrays, function signatures, and file I/O

Array essentials

Pattern	Syntax
Declare array	`int values[7];`
Initialize array	`int values[5] = {1,4,3,6,8};`
Indexing	`values[i]` valid only for legal indexes
Pass to function	`void printArray(const int list[], int size);`
Pointer equivalence	`list[i]` is the same as `*(list + i)`

Common array bugs

- For size 10, valid indexes are 0 through 9.
- Looping from 1 to 10 skips index 0 and writes past the end.
- When you pass an array, also pass its size.
- Do not assume uninitialized elements are zero.

Exam-style signatures

```
int match(const int one[], const int two[], int size);
int count(const char letters[], int size, char target);
bool search(const int data[], int size, int target);
void determineGPA(const double *const list, const int i, double &ave);
double *addNewClass(const double *const list, const double gpa, int &size);
```

Safe file I/O skeleton

```
int readExternalInfo(const char *inputFile) {
    ifstream input;
    input.open(inputFile);

    int sum = 0;
    int temp;
    while (input >> temp) {
        sum += temp;
    }

    input.close();
    return sum;
}
```

Why this loop? `while (input >> temp)` only enters when a read succeeds. It is safer than `while (!input.eof())`.

Part 4 - Dynamic arrays and heap patterns

Growing a heap array

```
double *addNewClass(const double *const list, const double gpa, int &size) {
    double *returnList = new double[size + 1];

    for (int i = 0; i < size; i++) {
        returnList[i] = list[i];
    }

    returnList[size] = gpa;
    size++;
    return returnList;
}
```

Caller usage pattern

```
double *tempGPA = addNewClass(gpaList, addedGPA, classCount);
delete [] gpaList;
gpaList = tempGPA;
```

new / delete pairing

Allocate	Free	Notes
`new int`	`delete p;`	single object
`new int[n]`	`delete [] p;`	heap array
`int *p = NULL;`	-	defined pointer value, no target yet

Leak: heap object still exists, but no pointer can reach it.
Dangling pointer: pointer still stores an address after its target was deleted.

Part 5 - Pointers, diagram-to-code, and memory reasoning

Pointer meaning table

Expression	Meaning
`int *p;`	p is a pointer to int.
`&x`	address of x
`*p`	value stored at the address in p
`p = &x;`	p now points to stack variable x
`p = new int;`	p now points to a heap int with unknown value until assigned
`*p = 88;`	write 88 into the pointee

Diagram -> code translation workflow

1. Create named stack variables first.
2. Create pointer variables next.
3. If the picture shows an anonymous heap box, use `new`.
4. Bind arrows with `&` or heap pointers.
5. Assign pointed-to values last.

Three comparison questions

Question	Test
Same pointee value?	`*P == *Q`
Same target address?	`P == Q`
Same storage location for the pointer vars?	`&P == &Q`

Worked patterns

```
// One pointer to a stack int
int foo = 365;
int *fooPtr = &foo;

// Two pointers alias the same target
int foo = 456;
int *fooPtr = &foo;
int *fooPtr2 = &foo;

// Pointer to heap int
int foo = 900;
int *fooPtr = new int;
*fooPtr = 88;
int *fooPtr2 = &foo;
```

Changing the caller's pointer

```
void pointerSwap(int *&p) {
    p = new int;
```

```
    *p = 33;
}
```

Why not `int *p`? That only changes a local copy of the pointer. The caller's pointer would still point to the old target when the function returns.

Part 6 - Pointer memory errors and exam-style mini patterns

How leaks and dangling pointers happen

Leak

```
int *p = new int;
*p = 88;
p = new int;   // old heap int lost -> leak
```

Dangling pointer

```
int *p = new int;
*p = 9021;
delete p;      // p still stores old address
-> dangling
```

Exam-style pointer traversal

```
int maxEntry(const int *const Data, int Sz) {
    if (Data == NULL || Sz <= 0) return INT_MIN;

    const int *hiSoFar = &Data[0];
    const int *Current = &Data[1];

    for (int Count = 0; Count < Sz - 1; Count++, Current++) {
        if (*Current > *hiSoFar)
            hiSoFar = Current;
    }
    return *hiSoFar;
}
```

Zeroing a heap array in the caller

```
void ZeroIt(int *&A, const int Size) {
    A = new int[Size];
    for (int i = 0; i < Size; i++) {
        A[i] = 0;
    }
}
```

Reason: the function must allocate and then make the caller's pointer point at the new array, so the parameter type must be `int *&`.

Part 7 - Curated example bank

Prototype + overload example

```
int max(int num1, int num2);
double max(double num1, double num2);
double max(double num1, double num2, double num3);
```

Pass-by-reference swap

```
void swap(int &n1, int &n2) {
    int temp = n1;
    n1 = n2;
    n2 = temp;
}
```

Pass array to function

```
void printArray(const int list[], int arraySize) {
    for (int i = 0; i < arraySize; i++) {
        cout << list[i] << " ";
    }
}
```

Read all ints from a file

```
void ReadData(const char *filename, int &size, int myArray[]) {
    ifstream inFile(filename);
    size = 0;
    int value;
    while (inFile >> value) {
        myArray[size] = value;
        size++;
    }
    inFile.close();
}
```

Practice-midterm readiness checklist

If the problem asks you to...	Look here first
fix syntax in a provided main / blank .cpp / half-finished .h	Parts 0, 1, 7
choose the correct prototype	Part 2 prototype decision table
read values from a file and sum / store them	Part 3 file I/O skeleton and Part 7 ReadData
grow an array and update size	Part 4 dynamic array pattern
recreate a memory diagram	Parts 5 and 6
change a caller's pointer	Part 5 pointerSwap and Part 6 ZeroIt

Tomorrow's add-on: after you finish the practice midterm, append the printed question packet and your corrected solutions as the last section.

Review (Material adapted from CS 2605 Fall 2006 at Virginia Tech)
EC327 Introduction to Software Engineering – Spring 2026; Rev 4.0

1. Which of the following is the proper declaration of a pointer?
A. int x;
B. int &x;
C. ptr x;
D. int *x;

2. Which of the following gives the memory address of integer variable a;?
A. *a;
B. a;
C. &a;
D. address(a);

3. Which of the following gives the memory address of a variable pointed to by pointer a?
A. a;
B. *a;
C. &a;
D. address(a);

4. Which of the following gives the value stored at the address pointed to by the pointer a?
A. a;
B. val(a);
C. *a;
D. &a;

For the next three questions, consider the following short program:

```
#include <iostream>
#include <new>
using namespace std;
int main() {
	char* pC = NULL; // Line 1
	pC = new char; // 2
	*pC = 'x'; // 3
	return 0;
}
```

5. Which of the following best describes the situation after Line 1 has been executed?
1) pC does not have a target, and pC does not have a defined value.
2) pC does not have a target, but pC does have a defined value.
3) pC has a target, but its target does not have a known value.
4) pC has a target, and its target does have a known value.
5) None of these

6. Which of the following best describes the situation after Line 2 has been executed?
1) pC does not have a target, and pC does not have a defined value.
2) pC does not have a target, but pC does have a defined value.
3) pC has a target, but its target does not have a known value.
4) pC has a target, and its target does have a known value.
5) None of these

7. Which of the following best describes the situation after Line 3 has been executed?
1) pC does not have a target, and pC does not have a defined value.
2) pC does not have a target, but pC does have a defined value.
3) pC has a target, but its target does not have a known value.
4) pC has a target, and its target does have a known value.
5) None of these

8. Assume the variable declarations:

```
int Foo = 0;
int *ptr = &Foo;
```

Which of the following statements will change the value of Foo to 1?
1) ptr++;
2) Foo++;
3) (*Foo)++;
4) (*ptr)++;
5) All of these
6) 1 and 2 only
7) 1 and 4 only
8) 2 and 4 only
9) 3 and 4 only
10) None of these

For questions 9 through 13, consider the following implementation of a function to find the maximum value in an array of integers. The for loop is required to be implemented using pointers to access elements rather than direct array indexing.

```
int maxEntry(const int* const Data, int Sz) { // Line 1
        if ( Data == NULL || Sz <= 0 ) return INT_MIN; // 2

        int Count = 0; // 3

        // Set hiSoFar to point to the first array element:
        const int *hiSoFar = __________; // 4

        // Set Current to point to the second array element:
        const int *Current =  __________; // 5

        for ( ; Count < Sz-1; _____________) { // 6
                if (_________) // 7
                hiSoFar = Current; // 8
        }

        return (__________); // 9
}
```

9. How should the blank in Line 4 be filled? (circle as many as you want)
1) Data
2) *Data
3) &Data
4) &Data[0]
5) Data[0]
6) 3 or 5 only
7) 3 or 4 only
8) None of these

10. How should the blank in Line 5 be filled? (circle as many as you want)
1) hiSoFar
2) hiSoFar++
3) Data++
4) &Data[1]
5) Data[1]
6) 2 or 4 only
7) 2 or 5 only
8) None of these

11. How should the blank in Line 6 be filled?
1) Count++
2) Current++
3) Count++, Current++
4) It should be left blank.
5) None of these

12. How should the blank in Line 7 be filled?
1) Current > hiSoFar
2) &Current > &hiSoFar
3) *Current > *hiSoFar
4) *Current < *hiSoFar
5) None of these

13. How should the blank in Line 9 be filled?
1) *hiSoFar
2) &hiSoFar
3) hiSoFar
4) It should be left blank.
5) None of these

For questions 14 through 16, assume that P and Q are pointers of the same type, and that each has been assigned a value.

14. What comparison would determine whether P and Q have targets with the same value?
1) &P == &Q
2) *P == *Q
3) P == Q
4) All of them
5) 1 and 2 only
6) 1 and 3 only
7) 2 and 3 only
8) None of these

15. What comparison would determine whether P and Q have the same target?
1) &P == &Q
2) *P == *Q
3) P == Q
4) All of them
5) 1 and 2 only
6) 1 and 3 only
7) 2 and 3 only
8) None of these

16. What comparison would determine whether P and Q are stored at the same memory location?
1) &P == &Q
2) *P == *Q
3) P == Q
4) All of them
5) 1 and 2 only
6) 1 and 3 only
7) 2 and 3 only
8) None of these

17. Consider implementing a function to dynamically allocate an array of integers and set all its elements to zero:

```
void ZeroIt( ____________ A, const int Size) {
        A = new int[Size];
        for (int Idx = 0; Idx < Size; Idx++) {
        A[Idx] = 0;
        }
}
```

Which of the following choices for the blank preceding the formal parameter A is best?
1) int*
2) int*&
3) const int*
4) int* const
5) const int* const
6) All of the above

For questions 18 through 21 assume that we have a dynamically allocated array A of integers of dimension Size, with memory layout as shown:

```
const int Size = 5;
int *A = new int[Size];
```

Index	Address
0	007D0E70
1	007D0E74
2	007D0E78
3	007D0E7C
4	007D0E80

18. Which code fragment(s) could be inserted in the blank in order to safely initialize each element of A to zero?

```
int* p = &A[0];
for (int Idx = 0; Idx < Size; Idx++, p++) {
        ____________________;
        }
```

1) *A = 0;
2) A[Idx] = 0;
3) *p = 0;
4) *Idx = 0;
5) All of the above
6) 1 and 2 only
7) 1 and 3 only
8) 2 and 3 only
9) 1 and 4 only
10) None of these

19. What value will be printed by the code fragment:

```
for (int Idx = 0; Idx < Size; Idx++) {
        A[Idx] = int(&A[Idx]); // typecast converts address to int
}
cout << hex << A[3] << endl; // manipulator causes hex output
```

1) 007D0E70
2) 007D0E74
3) 007D0E78
4) 007D0E7C
5) 007D0E80
6) Unknown
7) None of these

20. Assuming **only the initial declarations** given above, what logical error(s) would result if the following statement were executed: A = new int[2*Size];

1) A dangling pointer would result (a pointer whose value is the address of memory that the program no longer owns).
2) A memory leak would result (the program would own memory that it could no longer access).
3) Both a dangling pointer and a memory leak would result.
4) Neither a dangling pointer nor a memory leak, but some other logical error would result.
5) No logical error would result.

21. Assuming **only the initial declarations** given above, and execution of the correctly completed code given in question 18, what logical error(s) would result if the following statement were executed: delete [] p;

1) A dangling pointer would result (a pointer whose value is the address of memory that the program no longer owns).
2) A memory leak would result (the program would own memory that it could no longer access).
3) Both a dangling pointer and a memory leak would result.
4) Neither a dangling pointer nor a memory leak, but some other logical error would result.
5) No logical error would result.

Appendix A answers - Week 7 pointer review

Multiple choice key

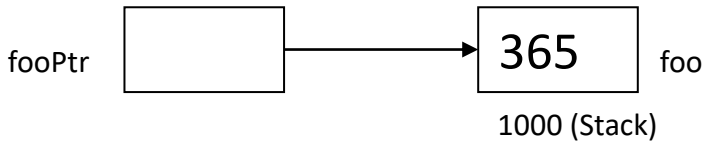
Q	Answer	Reason
1	D	`int *x;` declares a pointer to int.
2	C	`&a` is the address of variable `a`.
3	A	The pointer itself stores the target address, so the answer is `a`.
4	C	`*a` dereferences the pointer and gives the pointee value.
5	2	`NULL` is a defined pointer value, but it has no target.
6	3	After `new char`, `pC` has a target but the target value is not set yet.
7	4	After `*pC = 'x';`, both target and target value are known.
8	8	`Foo++` and `(*ptr)++` both change `Foo` to `1`.
9	1 and 4	`Data` and `&Data[0]` both point to the first element.
10	4	`&Data[1]` points to the second element without changing anything else.
11	3	Need to advance both the counter and the current pointer.
12	3	Compare pointee values, not addresses.
13	1	Return the maximum value, so dereference `hiSoFar`.
14	2	Same target value means `*P == *Q`.
15	3	Same target means the two pointer values match: `P == Q`.
16	1	Same memory location for the pointer variables means `&P == &Q`.
17	2	Need `int *&A` so the function can change the caller's pointer.
18	8	Both `A[Idx] = 0;` and `*p = 0;` are correct inside that loop.
19	4	`A[3]` receives the integer form of address `&A[3]`, which is 007D0E7C.
20	2	Reassigning `A` loses the old heap block: memory leak.
21	4	After the loop, `p` is one-past-the-end, so `delete [] p;` is an invalid delete (other logical error / UB).

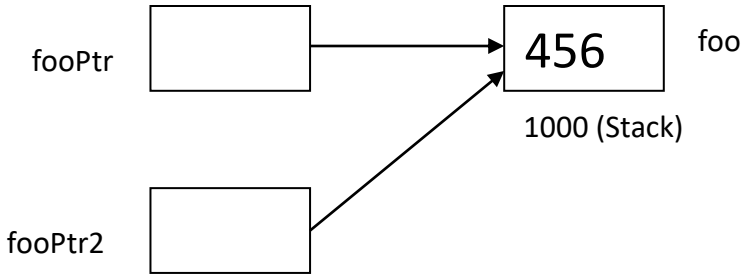
Most reusable ideas: `P == Q` means same target; `*P == *Q` means same value; `&P == &Q` means same storage location for the pointer variables; `int *&` is for rebinding a caller's pointer.

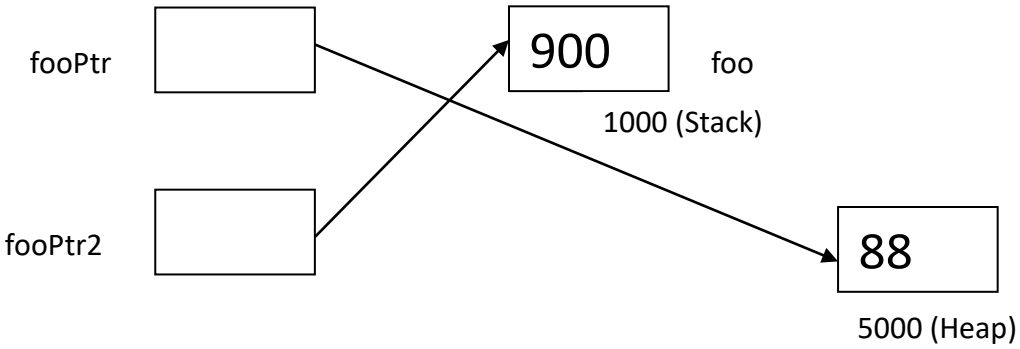
Worksheet Week 7 – More Pointers

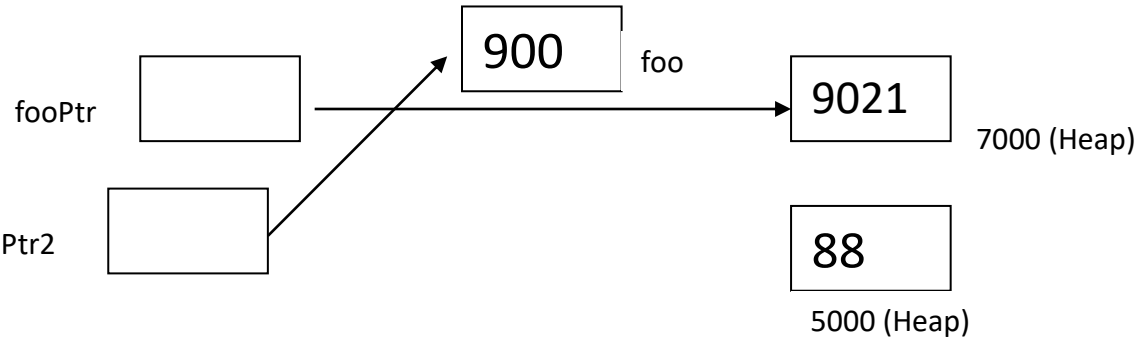
EC327 Introduction to Software Engineering – Spring 2026

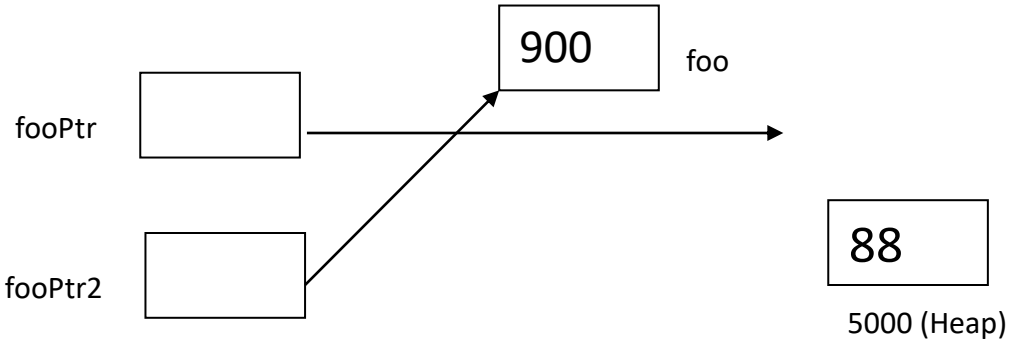
Write C++ snippets to create the following diagrams (memory address are assumed)
Fill in the boxes in the diagrams as much as possible
Highlight dangling pointers or memory leaks











Appendix B answers - Week 7 more pointers

Picture 1

```
int foo = 365;
int *fooPtr = &foo;
```

Pointer box value: **1000**. No leak. No dangling pointer.

Picture 2

```
int foo = 456;
int *fooPtr = &foo;
int *fooPtr2 = &foo;
```

Pointer box values: **1000** and **1000**. Two pointers alias the same target.

Picture 3

```
int foo = 900;
int *fooPtr = new int;
*fooPtr = 88;
int *fooPtr2 = &foo;
```

Pointer box values: **5000** and **1000**. Heap value 88 is still reachable, so no leak yet.

Picture 4

```
int foo = 900;
int *fooPtr = new int;
*fooPtr = 88;
int *fooPtr2 = &foo;
fooPtr = new int;
*fooPtr = 9021;
```

Now the old heap cell containing 88 has no remaining pointer to it. That is a **memory leak**.

Picture 5

```
int foo = 900;
int *fooPtr = new int;
*fooPtr = 88;
int *fooPtr2 = &foo;
fooPtr = new int;
*fooPtr = 9021;
delete fooPtr;   // fooPtr still stores the freed address
```

Dangling pointer: `fooPtr` still stores the old freed address.

Memory leak: the earlier heap cell with value 88 is still unreachable.

Worksheet Week 5 – Arrays
EC327 Introduction to Software Engineering – Spring 2026
Adapted from: http://cs.middlesexcc.edu/~schatz/csc133/worksheets

---

1. What is the output of the following program?

```
#include <iostream.h>
int main()
{
  int a[100], b[100], j, m;
  int suma = 0, sumb = 0, sumdiff = 0;
  cin >> m;
  for (j = 0 ; j < m ; j++)
  {
    cin >> a[j] >> b[j];
    suma = suma + a[j];
    sumb += b[j];
    sumdiff = sumdiff + (a[j] - b[j]);
  }
  for (j = m - 1 ; j >= 0 ; j--)
    cout << a[j] << "  " << b[j] << "  " << a[j] - b[j] << endl;
  cout << suma << "  " << sumb << "  " << sumdiff << endl;
}
DATA:
5
11 15
19 14
4 2
17 6
1 3
```

2. Given: int h = 6, p = 2, m = 3;
   int values[7];

   Suppose values contains: -4 0 2 6 -2 -1 14
   Show the contents of the array values after:

```
for (; m <=5; m++)
  values[m] = values[h] + values[p] * values[m];
```

3. Given the declarations:
   int sample[8], i, k;

   show the contents of the array sample after the following code is executed. Use a question mark to indicate any garbage values in the array.

```
for (k = 0 ; k < 8 ; k++)
  if (k % 2)
    sample[k] = 1;
```

4. What is the error in the following program segment?

```
int main()
{
  int i, count[10];
  cout << "please enter 10 numbers:  ";
  for (i = 1; i <= 10; i++)
    cin >> count[i];
}
```

5. Write the statements to multiply every element of an array of ints (of size 50) by 2.

6. Write the statements to add up those elements of an array of ints (of size 25) which have an even subscript.

7. Write the statements to add up those elements of an array of ints (of size 25) which have an even value.

8. What will the following program segment print?

```
int main()
{
  int nums[10];
  int i;
  for (i = 9 ; i >= 0 ; i--)
  {
    nums[i] = 5 * (i + 1);
    cout << nums[i] << "  ";
  }
  cout << endl;
  for (i = 0 ; i < 9 ; i++)
    cout << nums[i] << "  ";
  cout << endl;
  for (i = 0 ; i < 9 ; i++)
    nums[i+1] = nums[i];
  for (i = 0 ; i < 9 ; i++)
    cout << nums[i] << "   ";
  cout << endl;
}
```

9. What will the following program segment print?

```
int main()
{
  int nums[10];
  int i;
  for (i = 9 ; i >= 0 ; i--)
  {
    nums[i] = 5 * (i + 1);
    cout << nums[i] << "  ";
  }
  cout << endl;
  for (i = 0 ; i < 9 ; i++)
    cout << nums[i] << "  ";
  cout << endl;
  for (i = 8 ; i >= 0 ; i--)
    nums[i+1] = nums[i];
  for (i = 0 ; i < 9 ; i++)
    cout << nums[i] << "   ";
  cout << endl;
}
```

10. Given: int temps[50];
Write the statements to print "yes" if any element of the array temps contains the value 100.

11. Given: int temps[50];
Write the statements to set the variable found to true if any element of the array temps contains the value 100. If not, the variable found should be false.

Appendix C answers - Week 5 arrays

1. Output

```
1 3 -2
17 6 11
4 2 2
19 14 5
11 15 -4
52 40 12
```

2. Final contents of `values`

```
-4 0 2 26 10 12 14
```

3. Final contents of `sample`

```
? 1 ? 1 ? 1 ? 1
```

4. Error

The loop runs `i = 1` through `10`. That skips `count[0]` and writes to `count[10]`, which is out of bounds. Correct range: `for (i = 0; i < 10; i++)`.

5. Multiply every element by 2

```
for (int i = 0; i < 50; i++)
    arr[i] *= 2;
```

6. Sum elements with even subscript

```
int sum = 0;
for (int i = 0; i < 25; i += 2)
    sum += arr[i];
```

7. Sum elements with even value

```
int sum = 0;
for (int i = 0; i < 25; i++)
    if (arr[i] % 2 == 0)
        sum += arr[i];
```

8. Output

```
50 45 40 35 30 25 20 15 10 5
5 10 15 20 25 30 35 40 45
5 5 5 5 5 5 5 5 5
```

9. Output

```
50 45 40 35 30 25 20 15 10 5
5 10 15 20 25 30 35 40 45
5 5 10 15 20 25 30 35 40
```

10. Print yes if any element is 100

```
bool found = false;
for (int i = 0; i < 50; i++) {
    if (temps[i] == 100) {
        found = true;
        break;
    }
}
if (found)
    cout << "yes" << endl;
```

11. Set `found` appropriately

```
bool found = false;
for (int i = 0; i < 50; i++) {
    if (temps[i] == 100) {
        found = true;
        break;
    }
}
```

Worksheet Week 6 – Arrays Part II
EC327 Introduction to Software Engineering – Spring 2026
Adapted from: http://cs.middlesexcc.edu/~schatz/csc133/worksheets

---

1. Write a function match which takes 2 integer arrays (named one and two) and returns the number of times "matches" occur in parallel positions in the two arrays. That is, count the number of times one[i] == two[i]. The size of both arrays is the same. The arrays are passed as parameters along with their size.

2. Write a prototype for your function in question 1.

3. Write the statements to declare two arrays of size 25, read values into the arrays, call your match function, and print out the number of matches in the two arrays.

4. Write a function named count that will count and return the occurrences of a given character in an array named letters. The parameters will be the array letters, the size of the array, and the character to count.
   For example: If the array contained the values
   x 8 R A a a 0 s S a A
   and the character to count was 'a', then the function would return 3.

5. Write a prototype for the function in the previous problem.

6. Write the statements to declare an array of characters, and initialize the array to contain the characters: f A i @ N Z a 7 p Y h A. Call the count function and print the number of times the character p is contained in the array.

7. Write a function called search that receives an array of ints, the size of the array, and a number to search for. The function will return true if the number is contained in the array, and will return false otherwise.

8. Write a prototype for your search function.

9. Write the statements to call your function to search for the value 1500 in an array called salaries which has 100 elements. Print a message telling whether the value was found in the array.

Appendix D answers - Week 6 arrays part II

1-3. `match` function, prototype, and sample call

```
int match(const int one[], const int two[], int size) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (one[i] == two[i])
            count++;
    }
    return count;
}

int main() {
    int one[25], two[25];
    for (int i = 0; i < 25; i++) {
        cin >> one[i] >> two[i];
    }
    cout << match(one, two, 25) << endl;
}
```

4-6. `count` for character arrays

```
int count(const char letters[], int size, char target) {
    int total = 0;
    for (int i = 0; i < size; i++) {
        if (letters[i] == target)
            total++;
    }
    return total;
}

int main() {
    char letters[12] = {'f','A','i','@','N','Z','a','7','p','Y','h','A'};
    cout << count(letters, 12, 'p') << endl;
}
```

7-9. `search` and example use

```
bool search(const int data[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (data[i] == target)
            return true;
    }
    return false;
}

if (search(salaries, 100, 1500))
    cout << "1500 found" << endl;
else
    cout << "1500 not found" << endl;
```